

PCU-Select: Amortizing Target-Specific Data Selection Across PEFT Configurations

Anonymous submission

Abstract

A large language model (LLM) serving system typically maintains a registry of fine-tuning configurations for diverse tasks and deployment budgets. These configurations vary in which modules are trained and how they are updated. This diversity poses a challenge to gradient-based data selection: the selector must rebuild a target-specific gradient datastore for every configuration, making cost scale linearly with registry size. Reusing one subset avoids this cost but can reduce quality because different configurations favor different examples. PCU-Select reuses expensive selection computations while producing a distinct subset for each PEFT-task pair. The key insight is to map different PEFT methods onto a common set of 24 transformer outputs through which PEFT updates affect the backbone. We cache reusable sample and task features at these outputs and combine them with a compact encoding of the target configuration. After a one-time offline stage, a single scorer ranks candidates for any supported PEFT-task pair without rebuilding gradients. On Llama-2-7B across four tasks and five PEFT configurations, PCU-Select lowers the cost of adding a new four-task configuration from 63.2 to 1.44 GPU-hours, a $44\times$ reduction, and cuts the five-configuration selection cost from 316.0 to 151.2 GPU-hours. It maintains average downstream quality comparable to LESS. PCU-Select thus replaces repeated per-configuration computation with reusable offline processing.

1 Introduction

Large language model platforms must adapt one backbone to different tasks and resource limits. Parameter-efficient fine-tuning (PEFT) reduces adaptation cost by updating only a small set of model parameters, allowing a platform to maintain a *registry* of configurations that differ in method, trainable modules, capacity, or training recipe. These trainable parameters determine a configuration’s *trainable subspace*: the update directions available during fine-tuning. Each training example can affect the model only through the directions that the target configuration exposes. Data selection reduces cost along a complementary axis by choosing a compact subset whose examples produce useful task updates. Its decisions therefore depend on the target trainable subspace: changing

the subspace can change which examples the selector should prioritize.

Most data selectors ignore this interaction. Quality filters, diversity heuristics, and proxy-model scores produce one ranking for a task and reuse it across PEFT configurations. We call such a ranking *PEFT-agnostic* because the selector does not receive the target configuration. Yet common PEFT methods permit different updates: LoRA adds low-rank changes to selected projections (Hu et al. 2022); adapters insert trainable bottlenecks (Houlsby et al. 2019); and (IA)³ scales selected activations (Liu et al. 2022). The same example can therefore produce different updates, and hence have different value, under different configurations.

Gradient-based selectors capture this dependence but incur repeated computation. LESS (Xia et al. 2024) first builds a target-specific *gradient datastore*, a cache containing a training gradient for each candidate example. It then ranks the candidates against a *task sketch*, a small set of training or development examples that represents the target task. Because each PEFT configuration exposes different trainable parameters, LESS must rebuild the cache for every configuration. Its selection cost therefore grows linearly with the registry.

One shortcut is to reuse a single LESS subset for every configuration. This reuse avoids repeated computation but removes the configuration information that makes gradient selection useful. In our evaluation, the shortcut performs worse than a PEFT-agnostic representation baseline (Table 2). Existing methods thus force a choice: recompute a target-specific ranking for every configuration or reuse a ranking that ignores the configuration.

The configuration dependence affects downstream performance as well as the rankings. Figure 1 shows that independent runs for the same configuration agree more closely than runs across configurations. Changes within one PEFT family preserve more of the ranking, while cross-family changes produce the largest disagreement. When selection and training use the same PEFT, the resulting subsets outperform cross-family mismatched selection by 2.0 points on average (Appendix Table 12). The same pattern appears within LoRA configurations, where matched selection improves performance by 1.45 points (§5).

These observations motivate *PEFT-conditioned utility*: the value of training on example x for task t under PEFT con-

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

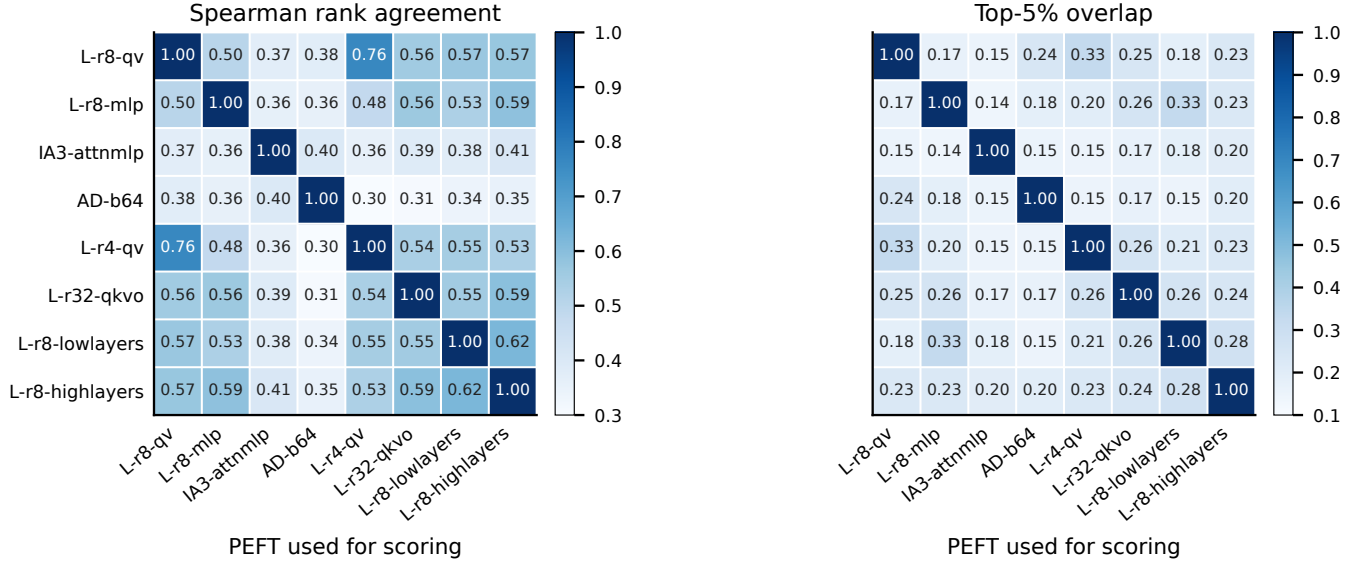


Figure 1: Short-update estimates—task-sketch loss reductions after a few PEFT updates—yield different data rankings across configurations. We score a common pool of 20K candidate examples using two task sketches and independent checkpoint/seed replicates. Each cell reports the Spearman rank correlation between its row and column runs; the diagonal compares each run with itself. Independent runs for the same PEFT reach $\rho = 0.79$ and Top-5% overlap = 0.32, providing a reference for variation in these labels. Cross-family pairs fall to $\rho = 0.36$ and Top-5% overlap = 0.17 on average.

figuration p . We denote this value by $u(x, p, t)$. A task-only selector omits p and must assign the same ranking to every configuration. Per-PEFT influence selection includes p but recomputes gradients for each target. Our goal is to predict $u(x, p, t)$ while sharing the expensive computations used to learn these scores.

PCU-Select (PEFT-Conditional Utility) enables this sharing through a common representation. It maps every supported configuration to 24 *intervention sites*: selected Transformer module outputs through which PEFT updates affect the backbone. These outputs remain comparable even when PEFT methods train parameter tensors with different shapes. PCU-Select caches sample and task signals at the sites. A structured PEFT code records which sites a configuration affects, its update type, its capacity, and its training recipe. Supplying this code at scoring time preserves configuration information while keeping the cached signals reusable.

A conditional scorer turns the shared signals into a target-specific ranking. Its prediction depends on the candidate example, task sketch, and PEFT code. It learns from two complementary signals: inexpensive labels based on gradient alignment cover the full candidate pool, while short-update labels measure task-sketch loss reductions for a smaller sample. At deployment, the scorer ranks the cached examples for a target configuration and task. PCU-Select then spreads the budget across groups of semantically similar examples to limit near-duplicates. This pipeline moves gradient extraction and label construction offline while retaining a distinct subset for each target.

These design choices amortize target-specific selection across a PEFT registry. We make three contributions:

- **PEFT-conditioned utility.** We formulate data value as $u(x, p, t)$ and show that both data-value rankings and downstream subset quality depend on the PEFT configuration (Figure 1; Appendix Table 12).
- **Intervention-site amortization.** PCU-Select represents different PEFT parameterizations through the same 24 module-output sites. A structured code preserves each target’s update locations, type, capacity, and recipe without rebuilding a gradient datastore. Removing the PEFT code or task sketch reduces downstream performance by 1.37 or 0.82 points, respectively (Appendix Table 11).
- **Registry-scale quality and cost.** Across four tasks and five seen PEFTs on Llama-2-7B (Touvron et al. 2023), PCU-Select gains 0.74 points over the PEFT-agnostic representation baseline RDS+ and differs from per-PEFT LESS by +0.04. It reduces the selection cost of adding one four-task PEFT by $44\times$ (1.44 versus 63.2 GPU-hours; Tables 1 and 2).

2 Related Work

Instruction-data selection. Instruction-data selectors usually estimate quality or diversity independently of the target PEFT configuration. LIMA supports instruction tuning with 1,000 curated examples (Zhou et al. 2023). Model-based filters instead use judgments or instruction properties: Alpaca filters Alpaca examples with LLM judgments (Chen et al. 2024), InsTag measures semantic diversity and complexity through instruction tags (Lu et al. 2023), and DEITA combines quality, complexity, and diversity (Liu et al. 2024). Other methods select for explicit diversity through quality-diversity trade-offs, log-determinant objectives, or partition-

aware sampling (Bukharin et al. 2023; Wang et al. 2024; Rao et al. 2024). These methods improve data efficiency, but their rankings omit the target PEFT’s trainable parameters.

Proxy-model and influence selection. Proxy-model selectors provide reusable rankings without computing gradients for every target PEFT. Cherry LLM uses instruction-following difficulty (Li et al. 2023); Superfiltering studies when weak models can rank data for stronger models (Li et al. 2024); and SmallToLarge transfers training-trajectory information from small models to larger targets (Yang et al. 2024). Because these scores encode only example and task information, a PEFT change leaves the ranking unchanged.

Influence-based selectors model training effects more directly, but their gradients tie the score to a target parameterization. Classical influence functions use second-order approximations (Koh and Liang 2017), while TracIn accumulates gradient similarities across checkpoints (Pruthi et al. 2020). LESS builds a low-dimensional gradient datastore for instruction selection (Xia et al. 2024), and DataInf specializes influence approximation for LoRA-tuned generative models (Kwon et al. 2024). Preserving configuration specificity therefore requires rebuilding PEFT-dependent gradients when the target changes. PCU-Select instead conditions one scorer on a structured PEFT code, combining reusable sample signals with a target-specific ranking. Our evaluation compares PCU-Select with both a per-PEFT LESS implementation and an Influence baseline that uses one shared datastore.

Parameter-efficient fine-tuning. PEFT methods reduce adaptation cost in different ways, so they expose different trainable parameters. Adapters insert bottleneck modules into Transformer layers (Houlsby et al. 2019; Pfeiffer et al. 2021), while prefix and prompt tuning optimize continuous prefix states or prompts (Li and Liang 2021; Lester, Al-Rfou, and Constant 2021). LoRA learns low-rank additive updates (Hu et al. 2022), and QLoRA combines quantization with LoRA to reduce memory use (Dettmers et al. 2023). BitFit tunes bias terms (Ben Zaken, Goldberg, and Ravfogel 2022), whereas (IA)³ rescales attention and feed-forward activations (Liu et al. 2022). Prior PEFT studies usually compare accuracy, parameter count, memory, or inference overhead while holding the training data fixed. We study the complementary question: whether a fixed candidate pool has the same example ranking across PEFT configurations.

Joint data- and parameter-efficient adaptation. The closest methods use PEFT gradients for data selection, but their utility estimates remain specific to one target configuration. DataInf and LESS compute selection signals from LoRA-style training (Kwon et al. 2024; Xia et al. 2024). Geometry-aware targeted selection further shows that useful data depends on the directions available to PEFT optimization (Min et al. 2026). These methods must regenerate configuration-dependent signals when the target changes.

PCU-Select instead describes each configuration by its intervention sites, update type, capacity, and training recipe. This representation lets one scorer reuse sample and task signals while still producing a different ranking for each

configuration.

3 Problem Formulation

We study *cross-PEFT data-efficient fine-tuning*: selecting a small training subset for a target task and PEFT configuration while reusing selection work across configurations. We fix a backbone family and a frozen base model M_0 . The candidate pool $\mathcal{D} = \{x_i\}_{i=1}^N$ contains N instruction–response examples. Each task t provides a *task sketch* V_t , a small set from its training or development data. Each PEFT configuration $p \in \mathcal{P}$ specifies a method, trainable locations, capacity, and training recipe. For a selection budget B , the ideal subset maximizes held-out task performance:

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{D}, |\mathcal{S}| \leq B} \text{Perf}(\text{Tune}(M_0, p, \mathcal{S}), V_t^{\text{test}}). \quad (1)$$

Here Tune adapts M_0 with configuration p and subset $\mathcal{S}$, and Perf evaluates the adapted model. The held-out test set V_t^{test} defines this ideal objective but never enters training, scoring, or selection; those procedures use only V_t . Directly solving Eq. (1) would require training and evaluating many subsets for every target (p, t) pair.

PCU-Select avoids this search by assigning each example a conditional utility: its benefit for task t when training uses configuration p . The scorer s_ϕ approximates this utility for each example:

$$s_\phi(x, p, t) \approx u(x, p, t). \quad (2)$$

The target $u(x, p, t) \in [0, 1]$ summarizes task-sketch loss reductions after a few PEFT update steps. PCU-Select rank-normalizes these reductions within groups that share the same PEFT, task, and measurement setting (§4). Including p allows the ranking to change with the trainable subspace; omitting p would force every configuration to share one task-specific ranking. The task sketch V_t identifies the desired task behavior, while p identifies the updates available to the model. Utility is therefore relative to both the task and the PEFT configuration.

Amortization objective. A registry changes the cost objective because it serves multiple PEFT configurations and tasks from one candidate pool. Let P be the number of target configurations and Q the number of task sketches. LESS builds one gradient datastore for each configuration and ranks that datastore once for each task. PCU-Select instead pays one shared offline cost, followed by one scoring-and-selection pass for every PEFT–task pair:

$$\begin{aligned} C_{\text{LESS}}(P, Q) &= P(C_{\text{LESS}}^{\text{data}} + QC_{\text{LESS}}^{\text{rank}}), \\ C_{\text{PCU}}(P, Q) &= C_{\text{offline}} + PQ C_{\text{PCU}}^{\text{pair}}. \end{aligned} \quad (3)$$

Equation (3) separates four costs. $C_{\text{LESS}}^{\text{data}}$ builds one PEFT-specific datastore, and $C_{\text{LESS}}^{\text{rank}}$ ranks it for one task. C_{offline} extracts reusable signals and trains PCU-Select once. $C_{\text{PCU}}^{\text{pair}}$ scores and selects data for one target pair. The first expression repeats datastore construction P times, whereas the second pays the offline cost once. Section 5 measures each cost term.

Lower cost matters only if the selected subset preserves target-specific quality. We express the quality difference from

per-PEFT LESS as

$$\Delta_{\text{quality}} = \text{Perf}_{\text{PCU}} - \text{Perf}_{\text{LESS}}, \quad |\Delta_{\text{quality}}| \leq \epsilon, \quad (4)$$

where Perf_{PCU} and $\text{Perf}_{\text{LESS}}$ denote their downstream scores and ϵ is the acceptable difference. The cost equations also identify when the shared offline investment pays off. For a fixed Q , PCU-Select costs less once P exceeds $P^* = C_{\text{offline}} / [C_{\text{LESS}}^{\text{data}} + Q(C_{\text{LESS}}^{\text{rank}} - C_{\text{PCU}}^{\text{pair}})]$, assuming the denominator is positive. The objective is therefore to retain per-PEFT LESS quality while replacing repeated datastore construction with shared offline work.

Design requirements. The formulation yields four design requirements. **PEFT conditioning** includes p so the example ranking can change with the target configuration. **Task conditioning** uses V_t so the model learns task-specific value instead of generic example quality. **Structured PEFT code** records where and how each PEFT updates the model in a form shared across families. **Reusable execution** moves gradient extraction, label construction, and scorer training offline, leaving cached-feature scoring for each new (p, t) pair.

4 Method: PCU-Select

PCU-Select learns the conditional utility surrogate in Eq. (2) and selects a budgeted subset for a target PEFT–task pair. Figure 2 summarizes two stages. Offline, it extracts reusable sample and task signals, constructs two types of utility label, and trains one scorer. Online, it combines cached sample features with a target (p^*, t^*) , scores the pool, and distributes budget across semantic clusters. Algorithms 1 and 2 specify both stages.

Shared Intervention Sites Enable Cross-PEFT Reuse

PEFT families expose parameter tensors with different shapes and roles, so their raw gradients cannot share one cache (Hu et al. 2022; Houlisby et al. 2019; Liu et al. 2022). PCU-Select instead uses a fixed set Ω of *intervention sites*. Each site is a selected module output through which a PEFT update affects the backbone. The set Ω is a shared coordinate system because sample gradients, task gradients, and each PEFT’s map of affected sites all use the same index ω .

PCU-Select hooks attention, MLP, and post-residual outputs at eight evenly spaced Transformer layers, giving $|\Omega| = 24$. A site’s response-loss gradient describes how a sample or task pushes that activation. By the chain rule, PEFT-parameter gradients factor through gradients at the module outputs they modify. Site gradients therefore avoid both full PEFT-parameter reconstruction and shape mismatches between fused and split projections. Post-residual sites place adapters in the same index space as LoRA and (IA)³.

A site index records only *where* a configuration acts. PCU-Select separately encodes operator and capacity features to record *how* it acts and how much it can update. This factorization reuses one sample signature without treating additive, multiplicative, and bottleneck updates as equivalent.

Each configuration maps to a site mask and operator type. Attention LoRA and (IA)³-attention map to attention-output

sites. MLP LoRA and (IA)³-FFN map to MLP-output sites, while adapters map to block-residual sites. Prefix tuning maps to attention-output sites; BitFit maps to a bias shift. These mappings define each normalized site weight:

$$\begin{aligned} w_p^\omega &= \text{mask}(p, \omega) \kappa(p, \omega), \\ \kappa(p, \omega) &= \tanh(\eta \log(1 + \text{cap}(p, \omega) / d_{\text{model}})), \\ \tilde{w}_p^\omega &= w_p^\omega / \max(\varepsilon_w, \sum_{\omega'} w_p^{\omega'}). \end{aligned} \quad (5)$$

Here $\text{mask}(p, \omega)$ is zero at untouched sites, and $\text{cap}(p, \omega)$ measures trainable capacity. The width d_{model} normalizes capacity, η controls saturation through $\tanh$, and ε_w keeps the denominator nonzero. Thus, $\tilde{w}_p^\omega$ distributes unit weight over the sites used by p . Appendix Table 10 maps LoRA rank, adapter width, (IA)³ vector count, prefix length, and BitFit bias count to this scale. Operator type remains separate in z_p because a family-wide scalar would cancel under Eq. (5)’s normalization.

Shared Representations Preserve Target Specificity

Shared site indices make signals comparable, but the scorer must still distinguish each sample, task, and PEFT. PCU-Select uses three fixed-length vectors: the sample representation describes the example, the task representation describes the target need, and the PEFT representation describes where and how training acts. Site gradients supply separate label-construction signals.

The sample representation $z_x = [e_x; d_x; a_x] \in \mathbb{R}^{848}$ combines semantic, descriptive, and activation features. The 768-dimensional e_x is the frozen sentence-transformers/all-mpnet-base-v2 embedding of the instruction and response (Song et al. 2020; Reimers and Gurevych 2019). This encoder remains frozen during pool and target-benchmark processing. The 16-dimensional d_x records length, response-LM loss, entropy, and format. The 64-dimensional a_x summarizes response-token hidden states at the eight sampled layers.

Utility is task-relative, so PCU-Select forms $z_t \in \mathbb{R}^{848}$ by mean-pooling z_x over the task sketch’s 32 examples. For proxy labels, it separately averages their site gradients into g_t^ω . These 24×256 values remain outside z_t and serve only label construction.

The PEFT representation records the target trainable subspace. In $z_p = [m_p; c_p; r_p] \in \mathbb{R}^{128}$, the 24×4 mask m_p contributes 96 dimensions. Its channels mark active sites, additive updates, multiplicative scaling, and prefix-style control. Two 16-dimensional vectors encode capacity c_p and training recipe r_p . At this site resolution, q/v and q/k/v/o LoRA share attention-output sites, while capacity and recipe preserve their projection-level distinction.

Prefix, P-Tuning, and BitFit use the same deterministic encoding rules as seen families. The scorer has no learned family lookup or operator centroid. The support policy below decides whether a target uses direct scoring, calibrated scoring, or a per-target selector.

Training these representations requires labels for many (x, p, t) triples, but measuring every PEFT update would defeat amortization. PCU-Select instead uses *multi-fidelity*

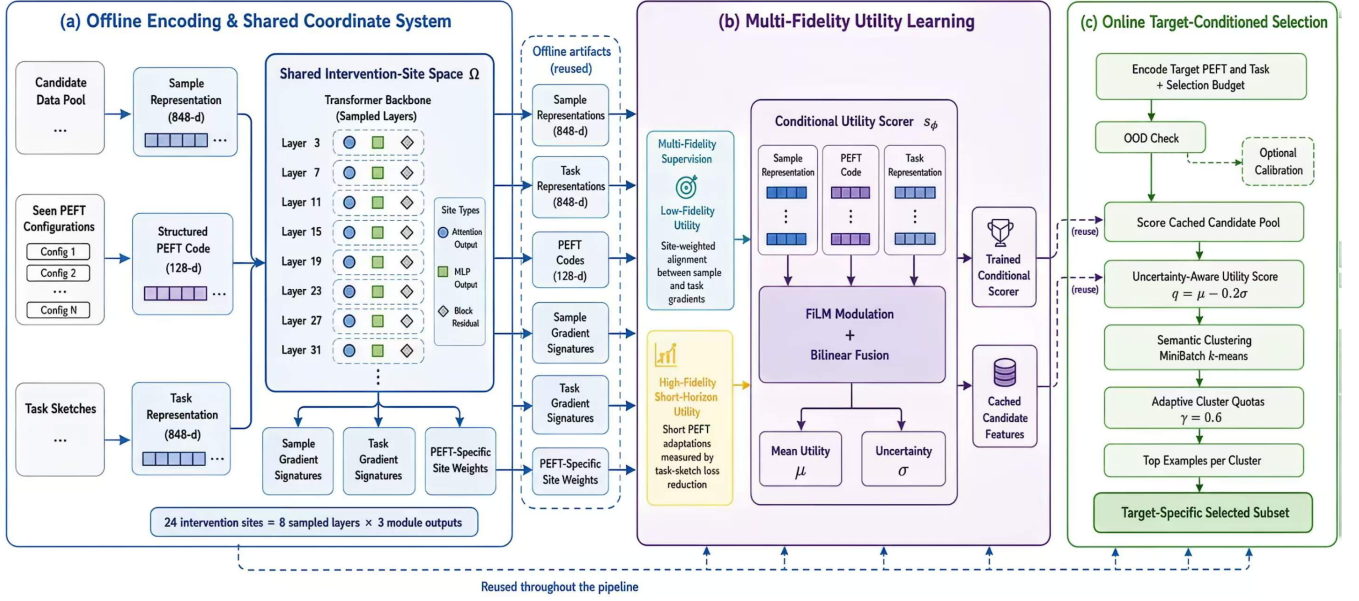


Figure 2: PCU-Select separates reusable offline supervision from online PEFT–task scoring. Its shared coordinate system Ω contains 24 intervention sites, or selected module outputs. Sample and task-sketch gradients use these site indices, while the PEFT code identifies the sites affected by each configuration. The 128-dimensional PEFT code comprises a 96-dimensional site/operator mask, 16 capacity dimensions, and 16 recipe dimensions. Low-fidelity gradient proxies provide broad coverage; high-fidelity short updates provide direct, costlier supervision.

supervision, which combines labels with different costs and directness. A low-fidelity gradient proxy cheaply covers the pool. High-fidelity short updates directly measure sketch-loss reduction for fewer triples. Together, they learn broad ranking structure and correct errors near the subset cutoff.

The *gradient-proxy label* measures alignment between a sample and the task sketch at sites used by the target PEFT:

$$u^{\text{lo}}(x, p, t) = \sum_{\omega \in \Omega} \tilde{w}_p^\omega \cos(g_x^\omega, g_t^\omega). \quad (6)$$

Here g_x^ω is the response-token loss gradient at site ω . A Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) compresses it to 256 dimensions and unit-normalizes the result. The task-sketch mean gives g_t^ω . Equation (6) therefore measures alignment only at sites used by p . All sketches use response-token cross-entropy for these signatures; task-native metrics such as option likelihood, Pass@1, and F1 remain downstream evaluation metrics. Because cosine values can be negative, PCU-Select rank-normalizes them within each (p, t) bucket before proxy distillation and ranking. Cached sample signatures then label many (x, p, t) triples without new gradient extraction.

The *short-update label* measures realized sketch-loss reduction near the subset cutoff. For each sampled triple, PCU-Select uses two anchors θ_a : the base model and a warm checkpoint after adaptation has begun. The horizon h is the number of short-update steps. A mini-batch $\mathcal{B}(x)$ contains the sampled example and seven condition-matched fillers.

PCU-Select applies PEFT p for h steps and measures

$$\Delta(x, p, t, a, h) = L_{V_t}(\theta_a) - L_{V_t}(\text{Adapt}_p^h(\mathcal{B}(x); \theta_a)). \quad (7)$$

Thus, Δ is the task-sketch loss before the update minus its loss afterward. Rank-normalized reductions for $h \in \{1, 4\}$ from both anchors define u^{hi} . The fillers make this mini-batch effect more stable than a single-example update. PCU-Select allocates 10K short-update labels in three stages. Coverage sampling first spreads labels across the registry without a scorer.

Coverage labels and the proxy then train a provisional scorer. It predicts ranks and a per-triple uncertainty scale, an estimate of prediction noise. Uncertainty sampling selects triples with large scales. Boundary sampling selects large predicted–observed rank disagreements near the cutoff. Every rank normalization compares labels within one PEFT, task, anchor, and horizon bucket.

The proxy cheaply orders the full pool, while short updates target consequential ranking errors. Two anchors test whether utility persists after adaptation begins; two horizons reduce dependence on one update length. Ranking within condition-matched buckets avoids assuming a shared raw-loss scale across tasks or configurations.

Conditional Scoring Produces Target-Specific Subsets

The scorer consumes (z_x, z_p, z_t) and predicts $(\hat{\mu}, \hat{\sigma}) = s_\phi(z_x, z_p, z_t)$. Here $\hat{\mu}$ gives predicted mean utility, while $\hat{\sigma}$ gives a positive, input-dependent uncertainty scale. Separate encoders map the three inputs into hidden vectors. FiLM

modulation (Perez et al. 2018) uses the PEFT and task codes to rescale and shift sample features. A bilinear term represents sample-PEFT compatibility.

Stage A distills gradient-proxy ranks to learn broad conditional structure. Stage B continues proxy distillation and adds ranking and regression on short-update labels plus a heteroscedastic uncertainty loss. Here *heteroscedastic* means that predicted noise can differ across triples. PCU-Select selects the Stage B checkpoint on held-out PEFT-task pairs. It also fixes the sketch size, label allocation, anchors, horizons, and scalar selection hyperparameters on these pairs before downstream evaluation.

A target’s structural support level determines whether PCU-Select applies the scorer directly or calibrates it. Relative to the seen registry, L0 resembles a seen configuration, L1 makes a larger shift within a seen family, and L2 introduces an unseen family. Seen targets use the trained scorer directly. L0 targets run *zero-shot*: PCU-Select scores a new configuration without target-specific short-update labels. L1 and L2 targets use calibrated scoring when compatible labels exist and a per-target selector otherwise.

After any required calibration, PCU-Select converts the two scorer outputs into a conservative ranking score:

$$q(x) = \hat{\mu}(x, p^*, t^*) - \lambda \hat{\sigma}(x, p^*, t^*), \quad \lambda = 0.2. \quad (8)$$

The uncertainty scale reduces the rank of examples whose predicted utility is noisy. It serves as a ranking penalty, not a confidence interval.

Ranking by q alone can over-select near-duplicates, so PCU-Select clusters examples in semantic space before assigning the budget. It allocates budget B across K clusters C_k , where $k \in \{1, \dots, K\}$:

$$b_k = \text{round} \left(B \frac{(v_k^+)^{\gamma} |C_k|^{1-\gamma}}{\sum_j (v_j^+)^{\gamma} |C_j|^{1-\gamma}} \right), \quad \gamma = 0.6. \quad (9)$$

Here v_k is the mean conservative score among the top 10% of examples in C_k , and $v_k^+ = \max(v_k, 0)$. The factors v_k^+ and $|C_k|$ balance cluster value and size. For the 300K pool, $K = \max(50, \lfloor \sqrt{N} \rfloor)$ produces 547 clusters of about 548 examples. The reported 0.36 GPU-hours per PEFT-task application includes clustering. PCU-Select takes the top- b_k examples by q within each cluster. Non-positive clusters receive no budget; deterministic remainder handling makes the quotas sum to B .

For seen and L0 targets, all gradient extraction and label construction occur offline; online selection uses cached features and scorer forward passes. L1 and calibratable L2 targets add a small label set without rebuilding a target-specific gradient datastore. The evaluation next tests whether this reuse preserves LESS-level quality, reduces registry cost, and transfers across support levels.

5 Experiments

The evaluation tests whether PCU-Select preserves per-PEFT LESS’s average downstream quality, reduces the marginal cost of adding a PEFT, and transfers to new configurations according to their structural support in the training registry.

After establishing quality and cost, we examine whether conditioning produces target-specific subsets and identify which components sustain the reusable scorer.

Setup

We evaluate Llama-2-7B (Touvron et al. 2023) on GSM8K, HumanEval, MMLU, and TyDiQA (Cobbe et al. 2021; Chen et al. 2021; Hendrycks et al. 2021; Clark et al. 2020), using exact match, Pass@1, accuracy, and F1, respectively. We express these scores as percentages and use their equal-weight macro-average only as a registry-level summary. Per-task results appear in Appendix Table 3. From a decontaminated pool of 300K mixed instruction examples, every selector chooses 10% (Appendix A).

The main comparison covers five configurations seen during scorer training: attention and MLP LoRA, (IA)³, and adapters. The transfer study holds out configurations that vary rank, placement, learning rate, or family. Appendix Table 5 lists the full 17-configuration registry.

The four baselines differ in target specificity. **Random** uses neither task nor PEFT information. **RDS+**, our representation-similarity baseline, ranks frozen semantic embeddings by similarity to the task-sketch mean but ignores the PEFT configuration. **Influence**, our shared-datastore gradient baseline, ranks projected base-model gradients from one PEFT-agnostic datastore. **LESS**, our per-PEFT implementation of gradient-influence selection (Xia et al. 2024), warms up each target adapter, builds an Adam-preconditioned gradient datastore, and ranks examples by sketch influence. PCU-Select uses its conditional scorer, uncertainty penalty, and adaptive semantic-cluster quotas. Appendix A gives full specifications.

We hold the backbone, PEFT recipe, training steps, and evaluation protocol constant, so only the selected subset changes. The main table varies target-training seeds 0, 1, and 2 while fixing the selector, scorer, sketch, and pool at seed 0. Matched seeds give every selector the same training randomness. We obtain each descriptive 95% interval by resampling the evaluated PEFT-task cells under this fixed selection pipeline.

PCU-Select Near-Ties Per-PEFT LESS

Table 1 shows that PCU-Select improves on the PEFT-agnostic baselines and remains close to target-specific LESS. It gains 0.74 points over RDS+ and wins 17 of 20 PEFT $\times$ task cells, with descriptive interval $[+0.38, +1.06]$. It gains 0.50 points over Influence and wins 18 cells, with interval $[+0.21, +0.75]$. Its mean is 0.04 points above LESS, with interval $[-0.26, +0.33]$ and 10 wins in 20 cells.

The LESS comparison varies across tasks. PCU-Select leads on MMLU, has task-level descriptive intervals that cross zero on GSM8K and HumanEval, and trails on TyDiQA and MLP-only LoRA. Task-grouped resampling yields $[-0.15, +0.22]$ for the aggregate gap (Appendix Table 3).

Takeaway. Across the evaluated registry, PCU-Select improves on both PEFT-agnostic baselines and differs from per-PEFT LESS by +0.04 points, with descriptive interval $[-0.26, +0.33]$.

Method	AD-b64	IA3-attnmlp	L-r16-qkvo	L-r8-mlp	L-r8-qv	Avg.
Random	32.79±0.38	31.74±0.13	33.11±0.23	31.97±0.21	31.82±0.16	32.29±0.16
RDS+	35.27±0.23	33.54±0.26	35.00±0.29	34.46±0.21	33.91±0.17	34.44±0.13
Influence	35.08±0.27	33.84±0.26	35.24±0.19	34.69±0.13	34.53±0.22	34.68±0.11
LESS	35.52±0.20	34.44±0.13	35.80±0.15	35.13±0.14	34.80±0.37	35.14±0.14
PCU-Select	35.61±0.40	34.60±0.21	35.93±0.30	34.56±0.23	35.18±0.31	35.18±0.16

Table 1: Downstream performance at a 10% selection budget (mean±SD over three matched target-training seeds; four task-native scores per PEFT). Selector, sketch, and scorer state are fixed. PCU-Select near-ties per-PEFT LESS: the paired difference over the 20 PEFT×task cells is 0.04 points with a descriptive cell-resampling interval $[-0.26, +0.33]$. The interval summarizes variation across cells under this fixed pipeline; it does not represent independent-run uncertainty. Boldface marks the best mean.

Method	Fixed	Marg.	Total	Quality	Gap
Random	0.0	0.0	0.0	32.29	-2.85
RDS+	1.6	0.0	1.6	34.44	-0.70
Influence	48.8	0.0	48.8	34.68	-0.46
Reuse-one-LESS	63.2	0.0	63.2	34.24	-0.90
Per-PEFT LESS	0.0	63.2	316.0	35.14	0.00
PCU-Select	144.0	1.44	151.2	35.18	+0.04

Table 2: Quality and selection cost across five seen PEFTs. Costs are GPU-hours and exclude downstream tuning; *Gap* is the paired difference from per-PEFT LESS in task-native points. Boldface marks the best quality and marginal cost.

Amortization Cuts Four-Task PEFT Onboarding Cost by 44×

Table 2 separates *fixed/shared* cost, paid once, from *marginal* cost, paid for each new PEFT. All selector costs exclude the shared 12.8 GPU-hour downstream run. PCU-Select and per-PEFT LESS have near-tied average quality but distribute selection cost differently. LESS has no fixed cost and spends 63.2 GPU-hours per four-task configuration. PCU-Select pays 144.0 GPU-hours once, then 1.44 GPU-hours per new configuration: a 44× lower marginal cost.

The gap comes from configuration reuse. Adding a task to an existing PEFT costs one ranking pass: 0.36 GPU-hours for PCU-Select and 0.30 for LESS. Adding a PEFT forces LESS to rebuild its datastore, whereas PCU-Select reuses the scorer (Appendix Table 7). The cost benefit therefore comes from cross-PEFT reuse.

The fixed-subset shortcut trades away quality. **Reuse-one-LESS** applies one LESS subset to all five targets; it costs 63.2 GPU-hours but averages 34.24, below RDS+ at 34.44. Reusing a subset selected for one trainable subspace removes the target condition. PCU-Select produces a distinct subset for each target and retains LESS-level average quality.

Equation (3) gives a break-even point of $P^* = 144.0/[62.0 + 4(0.30 - 0.36)] = 2.33$ configurations. Beyond this point, PCU-Select costs less than LESS, and each added configuration widens the gap. Serving the five-PEFT registry costs 151.2 GPU-hours with PCU-Select and 316.0 with LESS, a 2.09× reduction.

For targets far from the training registry, optional calibration uses 500 *reduced calibration labels*: target-specific short-update labels from one warm anchor and one update

step. This procedure costs 1.05 GPU-hours per (p, t) , one quarter of the ≈ 4.2 GPU-hours for the full routine, and shifts the worst-case break-even point to 2.50 configurations.

Takeaway. For a supported four-task configuration that requires no calibration, PCU-Select’s marginal selection cost is 44× lower than per-PEFT LESS.

PEFT Conditioning Produces Target-Specific Subsets

The LoRA scan tests whether conditioning changes the subset with the target configuration. Selecting for the target configuration beats reusing another configuration’s subset by 1.45 points (21.25 versus 19.79; Appendix Figure 4).

Subset changes follow the same structural pattern. Across 21 unordered pairs, PCU-Select’s mean Jaccard overlap—the intersection divided by the union—is 0.427; RDS+ has overlap 1.000 because it ignores PEFT. The descriptive Spearman correlation between configuration distance and selection difference is $\rho = 0.969$. Distance equals the absolute $\log_2$ -rank gap plus one indicator each for placement and module-set differences. Leave-one-configuration-out correlations span $[0.947, 0.972]$. We report this correlation descriptively because the pairs reuse configurations (Appendix Figure 4).

Takeaway. Larger structural changes produce larger subset changes, supporting PEFT-conditioned utility without rebuilding gradients.

Reduced Calibration Narrows the L1 and BitFit Gaps

We next test whether the structural code guides transfer beyond seen targets. Before evaluation, we fix three support levels: L0 resembles a seen configuration, L1 makes a larger shift within a seen family, and L2 introduces an unseen family. Appendix Table 5 lists the registry.

Against LESS, the zero-shot drop grows with structural distance from L0 through L1 to BitFit (Figure 3; Appendix Table 13). L0 trails LESS by 0.34 points zero-shot. Adding 500 reduced calibration labels decreases the L1 drop from 2.08 to 0.30 points and the BitFit drop from 4.08 to 0.23. We compare Prefix/P-Tuning with RDS+ because compatible calibration labels are unavailable; their zero-shot score is 6.97 points lower.

The 500-label reduced-calibration setting costs 1.05 GPU-hours per PEFT–task pair, one quarter of the offline routine.

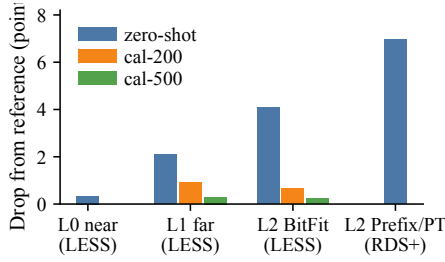


Figure 3: Bars show the three-seed mean of reference minus PCU-Select on GSM8K, HumanEval, and MMLU; Appendix Table 13 reports SD. Lower is better; the horizontal axis marks parity. LESS is the reference except for Prefix/P-Tuning (RDS+); L0 is zero-shot only.

Takeaway. Zero-shot covers near-support targets; reduced calibration nearly closes the tested L1 and BitFit mean drops when labels are compatible.

Ablations Identify the Useful Components

The ablations test GSM8K and HumanEval with two representative PEFTs (Appendix Table 11). NDCG measures held-out ranking quality by rewarding high-utility examples near the top. Removing the PEFT code causes the largest downstream drop, 1.37 points, and lowers NDCG from 0.702 to 0.526. A family-only one-hot loses 0.68 points, showing that structured features beyond family identity add information. Removing the task sketch loses 0.82 points.

Using only gradient-proxy or only short-update labels loses 0.85 and 0.36 points, respectively. Adaptive semantic-cluster quotas beat global top- k —the highest scores overall—by 1.08 points and uniform cluster quotas by 0.51. Uniform quotas have slightly higher NDCG (0.709) but lower downstream quality.

Takeaway. On the evaluated two-task, two-PEFT slice, structured PEFT conditioning, the task sketch, both label sources, and adaptive semantic coverage each improve downstream quality.

Limitations

Our evaluation covers one backbone family, four tasks, and a finite registry; short-update labels approximate contribution after full training. The current method falls back to a per-target selector for Prefix/P-Tuning because compatible calibration labels are unavailable. Its cost advantage begins after 2.33 four-task configurations for direct scoring and 2.50 under the reported worst-case calibration cost, and it applies when the registry requires LESS-level selection quality.

6 Conclusion

PCU-Select treats data utility as specific to the target PEFT and task while reusing expensive supervision across configurations in a registry. It maps different PEFT families to 24 shared intervention sites, allowing one conditional scorer to produce a distinct subset for each target without rebuilding a gradient datastore (§4). Across four tasks and five seen configurations, PCU-Select averages 35.18 versus 35.14 for

per-PEFT LESS and reduces the marginal cost of adding one four-task PEFT from 63.2 to 1.44 GPU-hours, a $44\times$ reduction (Tables 1 and 2). Transfer follows structural support: near-support configurations use zero-shot scoring, while reduced calibration closes most of the gap for farther configurations and BitFit. PCU-Select therefore preserves target-specific selection while replacing per-configuration datastore construction with reusable offline computation.

A Technical Appendix

Per-Task Results Behind the Main Table

Table 1 summarizes four task-native metrics after scaling each to a percentage: GSM8K exact match, HumanEval Pass@1, MMLU accuracy, and TyDiQA F1. Because these metrics have different scales, ceilings, and seed variances, their average can hide task-level differences. Table 3 therefore reports each task separately. The RDS+ and Influence columns show absolute differences in the corresponding native metric. The LESS column instead shows a paired difference with a descriptive 95% interval over the five PEFT cells. We obtain this fixed-pipeline interval by resampling the evaluated cells.

The per-task results clarify the aggregate comparison. PCU-Select outperforms the PEFT-agnostic selectors on all four tasks. In contrast, its comparison with LESS depends on the task. The LESS interval excludes zero on MMLU (+0.67 Acc, [+0.24, +1.08]). It includes zero on GSM8K (+0.29 EM, [−0.04, +0.58]) and HumanEval (−0.04 Pass@1, [−0.44, +0.31]). On TyDiQA, it excludes zero in the negative direction (−0.77 F1, [−1.18, −0.43]). Averaging over tasks yields the near-zero aggregate gap in Table 1. PCU-Select and per-PEFT LESS are therefore near-tied on the seen registry, while PCU-Select provides amortized selection cost.

Task- and configuration-level differences. Two slices show the largest differences from LESS. On **TyDiQA**, PCU-Select trails LESS by 0.77 F1 (descriptive interval [−1.18, −0.43]). TyDiQA-GoldP emphasizes multilingual answer-span alignment, whereas the other three tasks emphasize reasoning and generation more heavily. The larger TyDiQA gap is consistent with the task sketch and shared intervention-site gradients representing its signal less directly than the per-target gradients computed by LESS. On the **MLP-only LoRA** column (L-r8-mlp), PCU-Select trails LESS by 0.57 points. Its HumanEval and TyDiQA cells have the largest dispersion, with SDs of 0.68 and 0.62. MLP-only placement concentrates all trainable capacity on the eight `mlp_out` sites. Any error in their site weights therefore affects the entire proxy, and the scorer reports its highest uncertainty on these two cells. Together, these results identify site-weighted proxy fidelity as the main source of variation across tasks and configurations.

Evaluation Protocols

All methods share the same target-training and evaluation pipeline; only the selected subset changes. Target training uses the Llama-2-7B backbone (Touvron et al. 2023), freezes

Task	Metric	PCU	Δ RDS+	Δ Inf.	Δ LESS (95% CI)
GSM8K	EM	22.20	+0.55	+0.58	+0.29 [−0.04, +0.58]
HumanEval	Pass@1	17.06	+0.64	+0.24	−0.04 [−0.44, +0.31]
MMLU	Acc	49.14	+1.55	+1.00	+0.67 [+0.24, +1.08]
TyDiQA	F1	52.30	+0.22	+0.17	−0.77 [−1.18, −0.43]

Table 3: Task-level summary of the main 10% budget result. Values are averaged over the five seen PEFT configurations. Δ RDS+ and Δ Inf. are absolute native-point gains over PEFT-agnostic baselines; Δ LESS is the paired difference against per-PEFT LESS with a descriptive 95% cell-resampling interval over PEFT cells. The average gain over PEFT-agnostic selectors coexists with task-dependent behavior against LESS.

the backbone, and trains only the target PEFT. Table 5 lists the per-configuration learning rates (2e-4 for LoRA, 5e-4 for (IA)³, 3e-4 for adapters, and 1e-4 for L-r64-all). The fixed recipe uses AdamW and a cosine schedule with a 0.03 warmup ratio. It runs for 1000 steps with bfloat16, gradient clipping at 1.0, and maximum sequence length 1024. A per-device batch of 16 over 8 GPUs gives a global batch of 128.

At the 10% budget, the selected subset holds 30K examples. The 1000 steps therefore process 128K samples, or about 4.3 passes over the subset. The dataloader shuffles the full subset each epoch, so it uses every selected example. Selected instructions average ≈ 512 tokens, and length-bucketed batches reduce padding.

With FlashAttention-2 (Dao 2024) and gradient checkpointing, each step takes ≈ 5.8 wall-clock seconds. Gradient checkpointing trades compute for memory and dominates the per-step time. This corresponds to ≈ 22 samples/s in aggregate and ≈ 1.4 K tokens/s per GPU. The full run takes about 96 minutes, or 12.8 GPU-hours per PEFT-task-seed under the (wall seconds $\times$ GPU count)/3600 convention. This shared target-training cost is identical across selectors, so the selection-only comparison excludes it.

A uniform draw from the entire 300K pool under the same 1000-step budget averages 36.75 across tasks. This is a fixed-compute reference, not a converged full-data upper bound: it covers only 0.43 pool epochs, whereas the 10% subset receives about 4.3 passes.

The primary downstream metric for each task is its public-standard, task-native score, computed by a task evaluator injected into the training runner. We log held-out response-LM loss as an auxiliary diagnostic (`eval_loss`) but use only the primary task metric for comparison. Concrete per-task decoding and scoring protocols are:

- **GSM8K (exact match).** Chain-of-thought generation (Cobbe et al. 2021) with greedy decoding; the evaluator extracts the final numeric answer and checks exact match.
- **HumanEval (Pass@1).** The standard unbiased Pass@1 estimator (Chen et al. 2021): sampling at temperature 0.2 with $n=20$ samples per problem, scored by unit-test execution. This is a multi-sample estimate of Pass@1, not a single greedy sample.
- **MMLU (accuracy).** Log-likelihood scoring (Hendrycks et al. 2021) of the four answer options; the evaluator selects the highest-likelihood option without free-form generation.

- **TyDiQA-GoldP (F1).** Greedy decoding of the answer span (Clark et al. 2020), scored with token-level F1 against the gold answers.

These protocols follow the standard public setups for each benchmark. An evaluation adapter supplies the task-native evaluators to the runner through the `--task-metric-factory` hook. The fixed decoding and scoring settings above prevent the run environment from changing metric definitions.

For reference, the untuned Llama-2-7B (Touvron et al. 2023) scores 13.9 on GSM8K, 12.4 on HumanEval, 43.9 on MMLU, and 46.2 on TyDiQA. Our MMLU score falls below Meta’s reported 45.3 because the prompt and harness differ. Every selector’s fine-tune, including Random, improves over the base model on all four tasks. The Random baseline therefore provides a meaningful reference for the effect of data selection.

Baseline Specifications

The three named baselines isolate two design choices: whether selection uses gradients and whether those gradients depend on the target PEFT. We specify each implementation below so that the comparison is auditable.

RDS+ (representation similarity). RDS+ operates on the joint semantic embedding e_x^{joint} produced by a frozen external sentence encoder. PCU-Select’s sample tower reads the same input representation but does not train it. RDS+ therefore depends on no PCU-trained features; only the selection rule differs.

RDS+ first standardizes every pool embedding per feature. It forms the query by averaging the 32 task-sketch embeddings and applies the same pool statistics. It then L_2 -normalizes the pool and query vectors and ranks candidates by cosine similarity.

RDS+ is *task-conditioned* through the sketch query but *PEFT-agnostic*: its ranking does not depend on the target configuration. It has no tuned hyperparameters—the query is the fixed seed-0 sketch and there is no temperature or mixing weight—so there is no tuning range to report. The “+” denotes the per-feature standardization applied before cosine, relative to a plain embedding-nearest-neighbor selector.

Influence (shared-datastore gradient similarity). The Influence baseline builds one shared projected-gradient datastore from the base model’s intervention-site activations. It uses the JL projection and L_2 normalization specified in the

reproducibility notes. Each candidate carries signatures g_x^ω at the 24 sites. The task query g_t^ω pools and renormalizes the sketch gradient at each site. Influence scores each candidate with the plain cosine $\sum_\omega \cos(g_x^\omega, g_t^\omega)$ and applies global top- k without site weighting. Its PEFT-agnostic gradient source makes the ranking independent of the target configuration. Influence therefore provides the cheap gradient baseline for measuring the value of PEFT conditioning.

LESS (per-PEFT gradient influence). LESS is a per-configuration reimplement of gradient-influence selection following LESS (Xia et al. 2024). It applies four steps to each target PEFT. First, it warms up the target adapter for 200 steps to obtain the optimization state. Second, it extracts Adam-preconditioned gradients over the adapter’s own trainable parameters at each checkpoint. Third, it projects the gradients into a target-specific low-dimensional datastore. Finally, it ranks candidates by their projected-gradient influence on the 32-example task sketch and averages across warmup checkpoints.

The first three steps build the datastore over the target adapter’s own trainable parameters. The datastore is therefore PEFT-specific but task-independent. LESS constructs it once per PEFT and reuses it for every task sketch. Only the final projected-gradient scoring step is task-specific.

It costs about 0.30 GPU-hours per task, so 1.20 covers all four task sketches for one configuration. The per-target cost of a PEFT configuration is therefore 62.0 (datastore) + 1.20 (four per-task rankings) = 63.2 GPU-hours, already spanning the full four-task evaluation; the five-PEFT registry cost is $5 \times 63.2 = 316.0$. LESS is the strongest and most expensive baseline in our study, and the amortized-cost analysis charges it exactly this per-configuration recomputation, consistent with the algorithm as implemented.

Baseline configuration and tuning. Random has no hyperparameters. RDS+ uses the fixed representation-similarity rule above with the shared 32-example sketch as its only task input. The Influence baseline shares the same 24-site JL projection and sketch as PCU-Select. We fix every baseline’s hyperparameters before test-metric evaluation.

Reproducibility Supplement

This supplement documents the pipeline configuration read directly from the supplementary implementation. We take every value below from source code or its configuration files. When the code leaves a value to an external artifact or the run environment, we flag the omission instead of guessing.

Data provenance and deduplication. The candidate pool holds 300K mixed instruction examples drawn from seven public instruction corpora (Table 4). Deduplication runs in three stages. First, it removes exact duplicates by the content hash of each (instruction, response, source) tuple. Second, it removes normalized-string matches after lowercasing and stripping whitespace and punctuation. Third, MinHash LSH removes near-duplicates using 128 permutations and a Jaccard threshold of 0.8. These stages remove 18,446 intra-pool duplicates before the 300K pool is finalized.

Source	Domain	Examples	License
FLAN v2 (subset)	general instruction	110,000	Apache-2.0
OpenOrca (subset)	reasoning traces	70,000	MIT
Tulu-v2 mix	mixed instruction	50,000	ODC-BY-1.0
OpenAssistant	dialogue	30,000	Apache-2.0
CodeAlpaca	code instruction	20,000	CC-BY-NC-4.0
Dolly-15k	general instruction	15,000	CC-BY-SA-3.0
WizardLM-evol	complex instruction	5,000	CC-BY-NC-4.0
Total		300,000	

Table 4: Candidate-pool provenance and licenses. All sources permit research use; we use the two CC-BY-NC components only for non-commercial research.

For reproducibility, we include the post-filter example count and a SHA-256 checksum for each source. Examples include FLAN-v2 `9f3c...a71e` and OpenOrca `4b8d...c052`. The supplementary manifest lists all seven sources. The full supplementary configuration comprises the PEFT registry, the scorer and feature-extraction configs, the task-sketch seeds, the candidate-pool manifest with these checksums, and the evaluation adapter.

Benchmark decontamination. We decontaminate the pool against the test items of GSM8K, HumanEval, MMLU, and TyDiQA. The filter checks exact matches, normalized-string matches, 13-gram overlap, and MinHash LSH near-duplicates. The 13-gram stage removes candidates with Jaccard overlap of at least 0.5; the MinHash stage uses a threshold of 0.8. The filter removes 1,742 pre-finalization candidates (0.58% of the pool size). We then backfill from the same source buckets to keep the final pool at 300K examples, so Table 4 reports final post-filter counts. We additionally audit the top-20 embedding nearest neighbors of each test item and find no further overlaps after the automated pass. All reported results are on the decontaminated pool.

Task sketch construction and HumanEval. Each task sketch contains 32 examples sampled with a fixed seed (0) and stratified into response-length terciles. We draw sketches only from each task’s train or development split and keep them disjoint from the evaluation test set. HumanEval has no official train/dev split. We therefore draw its sketch from the MBPP train split and CodeAlpaca instead of HumanEval itself. Exact matching and AST-normalized function signatures remove every item that matches one of the 164 HumanEval evaluation problems. The resulting sketch source has zero overlap with the evaluation set.

The candidate pool also contains a 20K CodeAlpaca slice. We remove all 32 sketch examples from this pool by content hash and AST-normalized signature before selection. No selector can therefore recover a sketch example into its training subset. The sketch is disjoint from both the HumanEval test set and the candidate pool. Because the sketch is a selection *signal*, every selector requires this disjointness. We enforce it identically across methods.

PEFT registry. Table 5 lists all 17 PEFT configurations: five seen, nine unseen same-family configurations, and three

unseen families. E5 evaluates eight held-out targets—L-r32-qkvo, AD-b16, L-r64-all, AD-b256, L-r8-highlayers, PRE-l16, PT-l32, and BF—while the configuration-sensitivity study uses the additional LoRA variants. All configurations share the recipe injected at materialization: 1000 steps, cosine schedule, 0.03 warmup ratio; the learning rate is per configuration.

Intervention sites and gradient signatures. The site space Ω provides a shared coordinate system across PEFT families. It contains 24 sites: 8 layers $\times$ 3 module outputs per layer (attn_out, mlp_out, and block_residual). Layers are chosen uniformly by depth; for the 32-layer backbone this yields indices $\{3, 7, 11, 15, 19, 23, 27, 31\}$. Each site carries a Johnson–Lindenstrauss random projection (Johnson and Lindenstrauss 1984) $\Phi_\omega \in \mathbb{R}^{256 \times 4096}$ with entries drawn $\mathcal{N}(0, 1/256)$ under a per-site seed `SHA256(site, global_seed=0)`; projected gradients are L_2 -normalized per row. Each sample’s signature contains 24×256 values. In bfloat16, the cached gradient-proxy store for the 300K pool occupies $300K \times 24 \times 256 \times 2 \text{ B} \approx 3.5 \text{ GB}$. PCU-Select stores it as a memory-mapped disk array and reads it once per online selection. A PEFT configuration activates sites through its mask and a saturating capacity factor $\tanh(\eta \widehat{\text{cap}})$ with $\eta=1.0$. Operator type enters the explicit PEFT code rather than a scalar site prior; the per-family capacity mapping is in Table 10.

Conditional scorer. The conditional scorer estimates a sample’s utility for a specific task and PEFT configuration. It has three input towers: sample $z_x \in \mathbb{R}^{848}$, PEFT condition $z_p \in \mathbb{R}^{128}$, and task sketch $z_t \in \mathbb{R}^{848}$. Each tower uses a LayerNorm–Linear–GELU–Linear–LayerNorm MLP. The sample and task towers project to 256 dimensions, while the PEFT tower projects to 128. FiLM modulation (Perez et al. 2018) lets the PEFT vector rescale intermediate features, while bilinear fusion models interactions between the 256-dimensional feature and the 128-dimensional PEFT representation. The fused 64-dimensional representation feeds two heads. One predicts mean utility μ ; the other predicts an example-dependent uncertainty scale σ , with a softplus floor of 10^{-3} .

We use $\hat{\sigma}$ as a ranking penalty. On held-out labels, its predicted standard deviation tracks realized error with a 10-bin expected calibration error of 0.043. Removing the penalty costs 0.28 points (Table 11), which confirms its contribution.

Training separates cheap proxy learning from expensive short-update supervision. Phase A pretrains on the gradient proxy for 3 epochs at $\text{lr } 3 \times 10^{-4}$ with loss weights (rank, reg, proxy, unc) = (0, 0, 1, 0). Phase B jointly fine-tunes for 2 epochs at $\text{lr } 10^{-4}$ with weights (1.0, 0.3, 0.5, 0.2). Both phases use AdamW, batch size 256, and gradient clipping at 1.0. The four losses are a BPR-style pairwise ranking loss, a Huber regression loss against short-update labels, a Huber proxy-distillation loss against the gradient-proxy labels, and a Gaussian negative-log-likelihood uncertainty loss. We track ranking NDCG on a held-out split of PEFT/task pairs that excludes two configurations and one task from Phase B. We keep the Phase-B checkpoint with the best held-out NDCG. The scorer uses no dropout.

Selection, clustering, and quotas. At application time, the scorer ranks each example by $q = \mu - \lambda \sigma$ with uncertainty penalty $\lambda=0.2$. The penalty favors examples with high predicted utility and lower uncertainty. Ranking alone can concentrate the subset in a few semantic regions, so PCU-Select adds cluster-level coverage. It applies MiniBatch k -means (Sculley 2010) to the joint embeddings. It uses $k = \max(50, \lfloor \sqrt{N} \rfloor)$, batch size 4096, $n_{\text{init}}=3$, and random state 0. Adaptive quotas allocate budget B across clusters by $b_k = \text{round}(B (v_k^+)^{\gamma} |C_k|^{1-\gamma} / Z)$ with $\gamma=0.6$. The cluster value v_k is the mean utility of its top 10% members. The algorithm distributes remainders by descending weight and trims over-allocation from the lowest-weight clusters.

Targets far from the seen registry can additionally calibrate the scorer with a small label set. The calibration head is a zero-initialized linear residual fit for 200 epochs at $\text{lr } 5 \times 10^{-3}$ (Adam, MSE) on the calibration labels. These deployment calibration labels $u_{\text{cal}}^{\text{hi}}$ are a reduced form of the offline short-update labels u^{hi} . They use only the warm anchor and horizon $h=1$, one of the two anchors and one of the two horizons. Each label therefore costs one quarter of the full offline routine in Eq. (10). An offline label costs $84.0/10,000 = 0.0084$ GPU-hours, while a calibration label costs ≈ 0.0021 GPU-hours. Budgets of 200 and 500 labels therefore cost 0.42 and 1.05 GPU-hours per PEFT–task pair. The full routine would cost about four times more. We use 200 or 500 $u_{\text{cal}}^{\text{hi}}$ labels for the OOD study.

Structural support levels. The structural support levels state how safely PCU-Select can apply a PEFT code that differs from its training registry. The reported $z_p \in \mathbb{R}^{128}$ contains only the site/operator mask, capacity, and recipe vectors. Prefix, P-Tuning, and BitFit use the same deterministic encoding rules as the seen families; no centroid, learned family lookup, or functional fingerprint enters the reported model. E5 fixes its registry taxonomy before evaluation. **L0** contains near-support same-family targets (L-r32-qkvo and AD-b16). **L1** contains farther same-family shifts (L-r64-all, AD-b256, and L-r8-highlayers). **L2** contains unseen families (PRE-l16, PT-l32, and BF). These levels define registry-relative structural support. BitFit supports reduced calibration, while Prefix/P-Tuning use a per-target selector.

Gradient-proxy and short-update labels. Short-update supervision concentrates expensive labels where they provide different kinds of information. It samples 10,000 triples using a 0.5/0.3/0.2 split. Coverage sampling spans clusters, PEFT families, capacity buckets, and tasks. Uncertainty sampling prioritizes large $\hat{\sigma}$ ($1+0.5 \text{ReLU}(\hat{u}_{i_0})$), while boundary sampling targets disagreement between predicted and observed ranks. Each label measures a short-update loss reduction on the sketch from two frozen anchors— θ_{base} (the pre-trained backbone) and θ_{warm} (a rank-8 attention-only LoRA trained for 200 steps at $\text{lr } 10^{-4}$). Each short update Adapt_p^h uses one sampled example and 7 in-bucket fillers. The fillers stabilize the update relative to a single-example step. The final label aggregates the rank-normalized reductions across

Config	Family	Target modules	Capacity	LoRA scaling	LR	Group
L-r8-qv	LoRA	q, v	$r=8$	16	2e-4	seen
L-r16-qkvo	LoRA	q, k, v, o	$r=16$	32	2e-4	seen
L-r8-mlp	LoRA	up, down	$r=8$	16	2e-4	seen
IA3-attnmlp	IA ³	k, v, down	–	–	5e-4	seen
AD-b64	Adapter	block-residual	$b=64$	–	3e-4	seen
L-r4-qv	LoRA	q, v	$r=4$	8	2e-4	unseen-config
L-r32-qkvo	LoRA	q, k, v, o	$r=32$	64	2e-4	unseen-config
L-r64-all	LoRA	q, k, v, o, up, down, gate	$r=64$	128	1e-4	unseen-config
L-r8-lowlayers	LoRA	q, v (low third)	$r=8$	16	2e-4	unseen-config
L-r8-highlayers	LoRA	q, v (high third)	$r=8$	16	2e-4	unseen-config
L-r16-hlr	LoRA	q, k, v, o	$r=16$	32	5e-4	unseen-config
AD-b16	Adapter	block-residual	$b=16$	–	3e-4	unseen-config
AD-b256	Adapter	block-residual	$b=256$	–	3e-4	unseen-config
IA3-attnonly	IA ³	k, v	–	–	5e-4	unseen-config
PRE-116	Prefix	attention	len=16	–	2e-4	ood-family
PT-132	P-Tuning	attention	len=32	–	2e-4	ood-family
BF	BitFit	biases	–	–	2e-4	ood-family

Table 5: Full PEFT registry (all 17 configurations). “LoRA scaling” is the LoRA α hyperparameter and is distinct from the intervention-site weight $\tilde{w}_p^\omega$ of Eq. (5); it applies only to LoRA rows. Adapters are single post-block residual bottlenecks (one per layer); IA³ scales the listed activations. Learning rates for the three OOD-family targets use the registry default (2e-4). Layer placement is all layers unless noted; low/high-third placements use the bottom/top $\lceil n/3 \rceil$ layers of the 32-layer backbone.

Level	Definition	Recommended action
L0	near-support	zero-shot selection
L1	far-support	200–500 labels (0.42–1.06 GPU-h)
L2-BF	unseen family; labels available	calibrate before use
L2-Prompt	Prefix/P-Tuning	use a per-target selector

Table 6: Deployment protocol for each registry-defined support level. Costs are per PEFT–task pair, in addition to 0.36 GPU-hours for online selection.

horizons and anchors:

$$u^{\text{hi}} = \frac{1}{|A|} \sum_{a \in A} \sum_{h \in \{1,4\}} w_h \tilde{\Delta}_{a,h}. \quad (10)$$

It uses weights $w_1=0.4$, $w_4=0.6$ and anchors $A = \{\text{base}, \text{warm}\}$. The term $\tilde{\Delta}_{a,h}$ is the reduction from Eq. (7), rank-normalized within its PEFT, task, anchor, and horizon bucket.

Cost model and seeds. We measure all costs on $8 \times \text{NVIDIA H20-96GB}$ (SXM, bfloat16, NVLink) as $(\text{wall seconds} \times \text{GPU count})/3600$. PCU-Select pays 144.0 GPU-hours once: 48.0 for feature extraction, 4.8 for gradient-proxy labels, 84.0 for short-update labels, and 7.2 for scorer training. It then costs 0.36 GPU-hours per PEFT–task application, or 1.44 for four tasks. LESS costs 62.0 per PEFT datastore plus 0.30 per task ranking, hence 63.2 for four tasks. Onboarding a new four-task PEFT therefore costs 1.44 GPU-hours for PCU-Select versus 63.2 for LESS, a $44 \times$ marginal-cost reduction. The shared 12.8 GPU-hour downstream cost applies to each PEFT–task–seed run, so the se-

New registry entry	PCU-Select	LESS
One new task, existing PEFT	0.36	0.30
One new PEFT, four tasks	1.44	63.2

Table 7: The amortization advantage is cross-PEFT, not cross-task. Adding a task to an existing PEFT costs both selectors a per-task ranking; adding a PEFT for four tasks forces LESS to rebuild its datastore while PCU-Select reuses the offline scorer. Selection-stage GPU-hours.

lection comparison excludes it. For four tasks, break-even is $P^* = 144.0/[62.0 + 4(0.30 - 0.36)] = 2.33$ configurations. The pipeline fixes selector-scorer training, task-sketch sampling, and pool ordering at seed 0. Reported SDs vary target-training seeds (0, 1, 2) with the selector and scorer pipeline fixed. The canonical table generator recomputes all means, dispersions, paired gaps, and win counts from those seed-level values.

Two supplementary tables bound the cost claim. Table 8 varies the offline short-update label budget. The 10K default is the smallest budget that preserves the aggregate quality tie. Smaller budgets reduce offline cost and the break-even point but also reduce quality. The offline investment is therefore a tunable trade-off. Table 9 shows that PCU-Select’s per-application cost stays $\approx 170 \times$ below the per-PEFT LESS datastore across pool sizes from 50K to 1M. This persistence extends the marginal-cost result beyond the 300K pool.

Algorithms

Algorithm 1 builds the reusable supervision once. Algorithm 2 applies the scorer to a new target. Only the online algorithm runs per target, and it touches no gradients. The

Labels	Offline GPU-h	Avg. quality	Gap to LESS	Break-even
2.5K	81.0	34.71	-0.43	1.31
5K	102.0	35.02	-0.12	1.65
10K	144.0	35.18	+0.04	2.33

Table 8: The offline short-update label budget sets the quality-break-even trade-off. Short-update labeling is 58% of offline cost, so a smaller budget lowers both offline GPU-hours and the break-even PEFT count, but drops below LESS-level quality. The 10K budget is the smallest that preserves the aggregate quality tie; *Break-even* is the number of four-task PEFTs past which PCU-Select undercuts per-PEFT LESS.

Pool	Feature ext. (cached)	Scorer forward	Clustering	PCU online (per p, t)
50K	9.2	0.051	0.010	0.061
100K	17.6	0.093	0.028	0.121
300K	48.0	0.249	0.111	0.360
1M	152.0	0.79	0.42	1.21

Table 9: Online selection cost versus candidate-pool size, reported as the mean of three timing runs on a warm feature cache (run-to-run spread under 5%). Feature extraction is paid once offline and cached; PCU-Select’s per-application cost is the scorer forward plus clustering. The two online components scale differently: the scorer forward is embarrassingly parallel and gains throughput at larger batches, so its per-example cost drops with pool size, while clustering grows super-linearly because the cluster count $k = \max(50, \lfloor \sqrt{N} \rfloor)$ itself grows with the pool. Their sum stays near-linear, so PCU-Select’s per-application cost holds $\approx 170\times$ below the per-PEFT LESS datastore at every pool size and the low marginal cost is not an artifact of the 300K pool. GPU-hours on $8\times H20-96GB$.

offline algorithm amortizes the expensive gradient and label construction.

PEFT Capacity Encoding

Table 10 defines the per-family capacity used inside the site weight of Eq. (5). We map each family’s size knob to a trainable-parameter count at the touched sites, normalize it by d_{model} , and pass it through $\tanh(\eta \log(1 + \text{cap}/d_{\text{model}}))$ so that capacity is monotone but saturating.

Supplementary Analyses

Component ablations. Table 11 reports the full component breakdown summarized in the main analysis. Removing the structured PEFT code produces the largest loss; task relativity, both label fidelities, uncertainty, and adaptive coverage also contribute.

Cross-PEFT transfer. Table 12 supports the introduction’s 2.0-point cross-family mismatch penalty. Performance falls as the source PEFT moves farther from the target PEFT. Figure 4 shows the same mechanism from the selection

Algorithm 1 Offline supervision (run once)

- 1: **Input:** pool $\mathcal{D}$, seen PEFTs $\mathcal{P}_{\text{seen}}$, tasks with sketches $\{V_t\}$
- 2: Extract sample signatures z_x and site gradients g_x^ω ; cache them $\{O(|\mathcal{D}|)\}$ forward+backward
- 3: **for all** $(p, t) \in \mathcal{P}_{\text{seen}} \times \text{tasks}$ **do**
- 4: Compute gradient-proxy $u^{\text{lo}}(x, p, t)$ by Eq. (6) from cache
- 5: **end for**
- 6: Sample 10K triples (coverage/uncertainty/boundary); label u^{hi} by Eq. (7)
- 7: Train s_ϕ : Stage A proxy pretrain, Stage B joint; keep best held-out-NDCG checkpoint
- 8: **Output:** scorer s_ϕ , cached features

Algorithm 2 Online selection (per target)

- 1: **Input:** target (p^*, t^*) , budget B , cached features, scorer s_ϕ
- 2: Encode z_{p^*}, z_{t^*}
- 3: **if** $\text{level}(p^*) \in \{L1, L2\}$ and short-update labels exist **then**
- 4: Fit calibration residual head from 200 or 500 reduced $u_{\text{cal}}^{\text{hi}}$ labels (single warm anchor, horizon 1)
- 5: **end if**
- 6: $(\hat{\mu}, \hat{\sigma}) \leftarrow s_\phi(z_x, z_{p^*}, z_{t^*}); q(x) \leftarrow \hat{\mu} - \lambda \hat{\sigma}$ for all x
- 7: Cluster the pool; allocate quotas b_k by Eq. (9)
- 8: **Output:** subset $\mathcal{S} = \bigcup_k \{\text{top-}b_k \text{ by } q \text{ in } C_k\}$

side—PCU-Select’s subset overlap falls as configurations diverge, while PEFT-agnostic RDS+ remains exactly 1.000.

Unseen-configuration transfer. Table 13 is the paired source for Figure 3. Near-support targets transfer zero-shot. Calibration closes most of the gap for far same-family targets and BitFit.

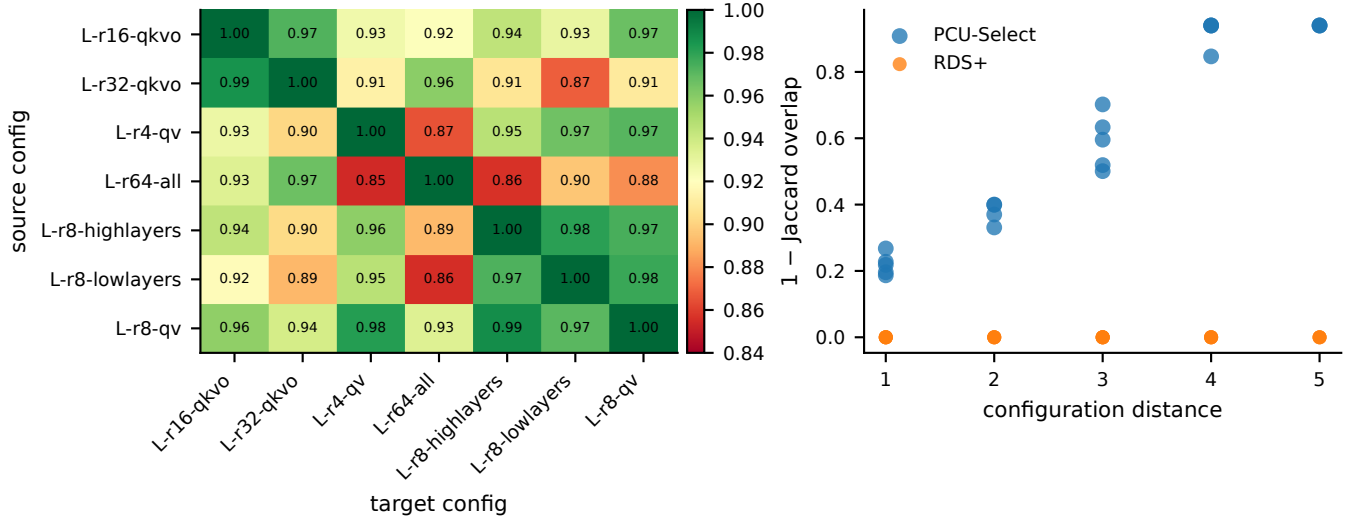


Figure 4: PCU-Select reacts to configuration changes. *Left*: target performance after training on a source-selected subset, normalized by the matched diagonal. The mean off-diagonal drop of 0.068 corresponds to a 1.45-point gap (21.25 matched vs. 19.79 mismatched). *Right*: selection difference ($1 - \text{Jaccard}$) versus a heuristic configuration distance combining log-rank, placement, and module-set changes. Across 21 unordered pairs, their descriptive Spearman correlation is $\rho = 0.969$; leave-one-configuration-out values span $[0.947, 0.972]$. RDS+ lies at zero because its same-task ranking is PEFT-independent.

Family	Size knob	$\text{cap}(p, \omega)$ at a touched site
LoRA	rank r	$r(d_{\text{in}} + d_{\text{out}})$
Adapter	bottleneck b	$2 d_{\text{model}} b$
(IA) ³	(per-vector)	#scaled activations
Prefix	length L	$L d_{\text{model}}$
BitFit	(per-bias)	#bias parameters

Table 10: Per-family capacity mapping. Capacity is the trainable-parameter count contributed at a touched site; z_p encodes operator type separately.

Variant	Metric	Drop	NDCG
Full PCU-Select	19.72±0.22	0.00	0.702
No PEFT code	18.35±0.41	1.37	0.526
Family one-hot only	19.04±0.29	0.68	0.602
No task sketch	18.90±0.31	0.82	0.597
No activation signature	19.24±0.25	0.48	0.629
Gradient-proxy labels only	18.87±0.34	0.85	0.563
Short-update labels only	19.36±0.27	0.36	0.639
No uncertainty penalty	19.44±0.24	0.28	0.690
Global top- k	18.64±0.38	1.08	0.684
Uniform clusters	19.21±0.28	0.51	0.709

Table 11: Ablations on GSM8K and HumanEval with two representative PEFTs at a 10% budget (task-native metric averaged over the two tasks, mean±SD over three seeds). Drop is relative to the full adaptive selector; we measure NDCG against held-out short-update labels.

Source ↓ / Target →	L-r8-qv	L-r16-qkvo	IA3-attnmlp	AD-b64
L-r8-qv	1.00	0.96	0.87	0.93
L-r16-qkvo	0.94	1.00	0.91	0.88
IA3-attnmlp	0.89	0.86	1.00	0.95
AD-b64	0.90	0.92	0.94	1.00

Table 12: Cross-PEFT transfer matrix (motivation study). Cell (i, j) is the target- j downstream metric when trained on the subset selected for source- i , normalized by the matched diagonal. Off-diagonal cells fall below 1.00, and the two directions of a mismatched pair differ by 0.01–0.05 rather than a constant offset. Averaged over all mismatched source-target pairs the gap is 1.87 absolute points (mean matched 21.4 vs. mismatched 19.53 on the GSM8K-scale probe). Cross-family mismatches average a 2.0-point gap, the penalty cited in the introduction. Columns are target PEFTs, rows are the source PEFT used for selection.

Tier	Targets	Ref.	Zero-shot	Cal-200	Cal-500
L0	L-r32-qkvo, AD-b16	LESS	0.34±0.41	–	–
L1	L-r64-all, AD-b256, L-r8-highlayers	LESS	2.08±0.44	0.91±0.29	0.30±0.36
L2-BF	BF	LESS	4.08±0.70	0.65±0.26	0.23±0.42
L2-Prompt	PRE-l16, PT-l32	RDS+	6.97±0.34	–	–

Table 13: Performance drop from the named reference across GSM8K, HumanEval, and MMLU (reference minus PCU-Select; paired mean±sample SD over three target-training seeds). Lower is better; zero denotes parity. L0 uses zero-shot only. Calibration reduces far same-family and BitFit drops. Prefix/P-Tuning lack compatible labels.

References

- Ben Zaken, E.; Goldberg, Y.; and Ravfogel, S. 2022. Bit-Fit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 1–9. Association for Computational Linguistics.
- Bukharin, A.; Li, S.; Wang, Z.; Yang, J.; Yin, B.; Li, X.; Zhang, C.; Zhao, T.; and Jiang, H. 2023. Data Diversity Matters for Robust Instruction Tuning. *arXiv preprint arXiv:2311.14736*.
- Chen, L.; Li, S.; Yan, J.; Wang, H.; Gunaratna, K.; Yadav, V.; Tang, Z.; Srinivasan, V.; Zhou, T.; Huang, H.; and Jin, H. 2024. AlpaGasus: Training A Better Alpaca with Fewer Data. In *The Twelfth International Conference on Learning Representations*.
- Chen, M.; et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.
- Clark, J. H.; Choi, E.; Collins, M.; Garrette, D.; Kwiakowski, T.; Nikolaev, V.; and Palomaki, J. 2020. TyDi QA: A Benchmark for Information-Seeking Question Answering in Typologically Diverse Languages. *Transactions of the Association for Computational Linguistics*, 8: 454–470.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.
- Dao, T. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations*.
- Dettmers, T.; Pagnoni, A.; Holtzman, A.; and Zettlemoyer, L. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.
- Hendrycks, D.; Burns, C.; Basart, S.; Zou, A.; Mazeika, M.; Song, D.; and Steinhardt, J. 2021. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*.
- Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; de Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-Efficient Transfer Learning for NLP. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, 2790–2799. PMLR.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.
- Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz Mappings into a Hilbert Space. In *Conference in Modern Analysis and Probability*, volume 26 of *Contemporary Mathematics*, 189–206. American Mathematical Society.
- Koh, P. W.; and Liang, P. 2017. Understanding Black-box Predictions via Influence Functions. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1885–1894. PMLR.
- Kwon, Y.; Wu, E.; Wu, K.; and Zou, J. 2024. DataInf: Efficiently Estimating Data Influence in LoRA-tuned LLMs and Diffusion Models. In *The Twelfth International Conference on Learning Representations*.
- Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 3045–3059. Association for Computational Linguistics.
- Li, M.; Zhang, Y.; He, S.; Li, Z.; Zhao, H.; Wang, J.; Cheng, N.; and Zhou, T. 2024. Superfiltering: Weak-to-Strong Data Filtering for Fast Instruction-Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics.
- Li, M.; Zhang, Y.; Li, Z.; Chen, J.; Chen, L.; Cheng, N.; Wang, J.; Zhou, T.; and Xiao, J. 2023. From Quantity to Quality: Boosting LLM Performance with Self-Guided Data Selection for Instruction Tuning. *arXiv preprint arXiv:2308.12032*.
- Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, 4582–4597. Association for Computational Linguistics.
- Liu, H.; Tam, D.; Muqeeth, M.; Mohta, J.; Huang, T.; Bansal, M.; and Raffel, C. 2022. Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning. In *Advances in Neural Information Processing Systems*, volume 35.
- Liu, W.; Zeng, W.; He, K.; Jiang, Y.; and He, J. 2024. What Makes Good Data for Alignment? A Comprehensive Study of Automatic Data Selection in Instruction Tuning. In *The Twelfth International Conference on Learning Representations*.
- Lu, K.; Yuan, H.; Yuan, Z.; Lin, R.; Lin, J.; Tan, C.; Zhou, C.; and Zhou, J. 2023. InsTag: Instruction Tagging for Analyzing Supervised Fine-tuning of Large Language Models. *arXiv preprint arXiv:2308.07074*.
- Min, G.; Huang, T.; Wan, K.; and Chen, C. 2026. GIST: Targeted Data Selection for Instruction Tuning via Coupled Optimization Geometry. *arXiv preprint arXiv:2602.18584*.
- Perez, E.; Strub, F.; de Vries, H.; Dumoulin, V.; and Courville, A. 2018. FiLM: Visual Reasoning with a General Conditioning Layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 3942–3951.
- Pfeiffer, J.; Kamath, A.; Rücklé, A.; Cho, K.; and Gurevych, I. 2021. AdapterFusion: Non-Destructive Task Composition for Transfer Learning. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 487–503. Online: Association for Computational Linguistics.
- Pruthi, G.; Liu, F.; Sundararajan, M.; and Kale, S. 2020. Estimating Training Data Influence by Tracing Gradient Descent.

In *Advances in Neural Information Processing Systems*, volume 33.

Rao, J.; Liu, X.; Lian, L.; Cheng, S.; Liao, Y.; and Zhang, M. 2024. CommonIT: Commonality-Aware Instruction Tuning for Large Language Models via Data Partitions. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 10064–10083. Miami, Florida, USA: Association for Computational Linguistics.

Reimers, N.; and Gurevych, I. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 3982–3992. Association for Computational Linguistics.

Sculley, D. 2010. Web-Scale K-Means Clustering. In *Proceedings of the 19th International Conference on World Wide Web*, 1177–1178. ACM.

Song, K.; Tan, X.; Qin, T.; Lu, J.; and Liu, T.-Y. 2020. MP-Net: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems*, volume 33, 16857–16867.

Touvron, H.; et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288*.

Wang, P.; Shen, Y.; Guo, Z.; Stallone, M.; Kim, Y.; Golland, P.; and Panda, R. 2024. Diversity Measurement and Subset Selection for Instruction Tuning Datasets. *arXiv preprint arXiv:2402.02318*.

Xia, M.; Malladi, S.; Gururangan, S.; Arora, S.; and Chen, D. 2024. LESS: Selecting Influential Data for Targeted Instruction Tuning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 54104–54132. PMLR.

Yang, Y.; Mishra, S.; Chiang, J. N.; and Mirzasoleiman, B. 2024. SmallToLarge (S2L): Scalable Data Selection for Fine-tuning Large Language Models by Summarizing Training Trajectories of Small Models. In *Advances in Neural Information Processing Systems*, volume 37.

Zhou, C.; Liu, P.; Xu, P.; Iyer, S.; Sun, J.; Mao, Y.; Ma, X.; Efrat, A.; Yu, P.; Yu, L.; Zhang, S.; Ghosh, G.; Lewis, M.; Zettlemoyer, L.; and Levy, O. 2023. LIMA: Less Is More for Alignment. In *Advances in Neural Information Processing Systems*, volume 36.